Wavelet Convolutions Are Memory-Bound: An I/O-Optimal Formulation of WTConv

Anonymous authors
Paper under double-blind review

Abstract

Wavelet convolution (WTConv) attains a receptive field that grows exponentially in the number of decomposition levels while its parameter count grows only linearly, and it drops into standard backbones without architectural change. Its reference implementation is nevertheless slower than the dense 7×7 convolution it is meant to replace, which has kept an asymptotically attractive operator practically unattractive. We show that this gap is not a constant-factor engineering artifact but a direct consequence of how the operator is expressed: WTConv performs $\mathcal{O}(1)$ arithmetic per element at an arithmetic intensity of 0.4–0.9 FLOP/byte, roughly two orders of magnitude below the balance point of a modern GPU, so its cost is determined almost entirely by data movement between stages that each round-trip through high-bandwidth memory. We give an exact I/O-complexity accounting of the operator, deriving that the reference form moves $7N + \frac{43}{3}N(1 - 4^{-L})$ elements against a lower bound of $4N + \frac{13}{3}N(1 - 4^{-L})$ for an input of $N = B \cdot C \cdot H \cdot W$ elements and L levels, and we close that gap with three exact algebraic reformulations: the Haar analysis cascade is recomputed in registers rather than materialised, which is free because the Haar butterfly is multiplication-free; the L-level synthesis cascade collapses into a single closed-form pass whose sign pattern is indexed by the bits of the output coordinate, eliminating every intermediate low-pass plane; and the learned per-channel scale is folded into the convolution weights by linearity. Self-adjointness of the normalised Haar operator ($\mathbf{H} = \mathbf{H}^\top = \mathbf{H}^{-1}$) makes each direction the other's gradient, so the backward pass requires no additional kernels and is exact rather than approximate. The resulting formulation is a bit-exact reassociation of the original operator, not an approximation. Our I/O model predicts a 2.45–2.56× speedup with no fitted parameters; a CUDA realisation measures 2.24–2.87× in `fp32` and up to 3.84× in `fp16`, at 1.5× lower peak memory, with training curves indistinguishable from the reference. Because the argument is about data movement rather than any hardware feature, the same formulation transfers unchanged to Triton and to Apple Metal.

1 Introduction

Convolutional networks acquire large receptive fields slowly. Stacking 3×3 kernels grows the theoretical receptive field linearly in depth and the *effective* receptive field only as $\mathcal{O}(\sqrt{n})$ (Luo et al., 2016), which is a large part of why vision transformers, whose first layer already mixes globally (Dosovitskiy et al., 2021; Liu et al., 2021), were initially so competitive. The convolutional response has been to enlarge kernels directly: 31×31 re-parameterised depthwise kernels (Ding et al., 2022), or 51×51 sparse factorised ones (Liu et al., 2023). Both buy receptive field at a parameter and FLOP cost that grows with the kernel area.

WTConv (Finder et al., 2024) takes a different route. It applies a small $k \times k$ depthwise kernel independently at each level of an L-level Haar decomposition. Because level ℓ operates on a 2^ℓ-times-downsampled grid, its footprint in input pixels is $k \cdot 2^\ell$, so an L-level layer reaches $k \cdot 2^L$ pixels using $\mathcal{O}(Lk^2C)$ parameters: the receptive field grows exponentially in L while parameters grow linearly, i.e. parameters grow only logarithmically in the receptive field. It is a drop-in replacement for depthwise convolutions and improves accuracy and robustness in ConvNeXt and MobileNetV2 backbones (Liu et al., 2022; Sandler et al., 2018).

There is a catch that the asymptotics hide. In the reference implementation, a WTConv layer is *slower than the dense* 7×7 *convolution it is supposed to replace* — by 1.14–1.65× in our measurements. An operator that is cheaper in parameters and FLOPs but more expensive in wall-clock time is a poor trade in practice, and this is the gap we close.

The cost of WTConv is data movement, not arithmetic. Our starting observation is that WTConv is nowhere near compute-bound. Counting the multiply–accumulates the operator actually performs against the bytes its reference implementation actually moves gives an arithmetic intensity of 0.44–0.89 FLOP/byte (Section 3). An NVIDIA RTX A6000 balances at roughly 50 FLOP/byte. The operator therefore sits two orders of magnitude to the left of the roofline ridge: the GPU spends essentially all of its time waiting for memory, and the arithmetic — including the Haar transform itself — is free in comparison. This reframes the problem. The question is not how to compute the wavelet transform faster; it is how to stop writing intermediate tensors to high-bandwidth memory (HBM). The same reframing drove FlashAttention (Dao et al., 2022), and the analogy is close: in both cases a mathematically elegant operator was expressed as a chain of memory-bound primitives, and in both cases the fix is a reassociation that keeps intermediates on chip.

What makes WTConv especially amenable. Two structural properties of the Haar basis make the reassociation unusually clean, and they are what we exploit.

The Haar butterfly is multiplication-free. The 2×2 Haar analysis of a patch $[[a, b], [c, d]]$ is four sums and differences scaled by $\frac{1}{2}$. Recomputation — normally the price one pays to avoid materialisation — is therefore almost free here, whereas for a general wavelet or a general transform it would not be. This turns the usual recompute/materialise trade-off into a strict win.

The normalised Haar operator is self-adjoint and involutive. With the $\frac{1}{2}$ normalisation, the 4×4 analysis matrix satisfies $\mathbf{H} = \mathbf{H}^\top = \mathbf{H}^{-1}$ exactly (Proposition 3). Consequently the backward pass of the analysis *is* the synthesis, and the backward pass of the synthesis *is* the analysis. No separate gradient kernels exist, and gradient correctness is a theorem rather than a test.

Contributions.

- **A cost model for WTConv** (Section 3). We give a closed form for the operator that eliminates the recursive low-pass carrier, and an exact element-by-element I/O accounting of the reference implementation: $Q_{\text{ref}} = 7N + \frac{43}{3}N(1 - 4^{-L})$. We place the operator on the roofline and show it is bandwidth-bound by two orders of magnitude.
- **An I/O-optimal reformulation** (Section 4) built from three exact algebraic identities — register-resident analysis, closed-form single-pass synthesis, and scale folding — reaching $Q_{\text{fused}} = 4N + \frac{13}{3}N(1 - 4^{-L})$, within a small constant of the $2N$ compulsory traffic. The synthesis collapse (Section 4.3) is the main algorithmic step: an L-level inverse cascade, normally L dependent passes, becomes one pass whose per-level sign pattern is read off the bits of the output coordinate.
- **A parameter-free prediction, and its validation** (Section 6). The model predicts 2.45–2.56× with nothing fitted. A CUDA realisation measures 2.24–2.87× (`fp32`), agreeing to within 12% across L, which is strong evidence that the I/O account — not incidental tuning — explains the gain.
- **Exactness, not approximation.** Every step is a reassociation of the same multilinear map, so agreement with the reference is limited only by floating-point reassociation error, which we quantify for the forward pass and for every gradient the layer produces.

We develop the argument for a single CUDA realisation to keep the exposition concrete. Because the reformulation is about data movement and not about any hardware feature, it transfers unchanged; we summarise Triton and Apple Metal realisations in Section 7 and Section C.

2 Background

2.1 The WTConv operator

Let $\mathsf{X} \in \mathbb{R}^{B\times C\times H\times W}$ be the input, over a batch of B images with C channels at resolution $H \times W$, and let $N = B \cdot C \cdot H \cdot W$ denote its number of scalar entries. All I/O costs below are quoted as multiples of N, so a cost of, say, $7N$ means "seven times the input tensor's worth of data". Expressing costs this way is what makes the speedup prediction of Section 4.6 a pure ratio, independent of B, C, H and W. WTConv is channel-preserving and depthwise. Let $\mathcal{A}$ denote one level of 2×2 Haar analysis, mapping $B \times C \times H \times W$ to $B \times C \times 4 \times \frac{H}{2} \times \frac{W}{2}$ with subbands ordered $(\mathrm{LL}, \mathrm{LH}, \mathrm{HL}, \mathrm{HH})$, and $\mathcal{A}^\top$ the corresponding synthesis. Let $\mathbf{W}^{(\ell)} \in \mathbb{R}^{4C\times 1\times k\times k}$ and $\mathbf{s}^{(\ell)} \in \mathbb{R}^{4C}$ be the level-ℓ depthwise kernel and learned per-channel scale, and write

$$\mathcal{C}_\ell(\mathsf{Y}) \;=\; \mathbf{s}^{(\ell)} \odot \big(\mathsf{Y} *_{\mathrm{dw}} \mathbf{W}^{(\ell)}\big). \tag{1}$$

The reference implementation (Finder et al., 2024) runs an analysis loop that carries the *raw* (unconvolved) low-pass band forward, $\mathsf{X}^{(0)} = \mathsf{X}$ and $\mathsf{X}^{(\ell)} = \big[\mathcal{A}\mathsf{X}^{(\ell-1)}\big]_{\mathrm{LL}}$, and a synthesis loop that accumulates across levels, $R^{(L+1)} = 0$ and

$$R^{(\ell)} \;=\; \mathcal{A}^\top\Big(\big[\mathcal{C}_\ell(\mathcal{A}\mathsf{X}^{(\ell-1)})\big]_{\mathrm{LL}} + R^{(\ell+1)} \,\|\, \big[\mathcal{C}_\ell(\mathcal{A}\mathsf{X}^{(\ell-1)})\big]_{\mathrm{H}}\Big), \tag{2}$$

with a full-resolution additive base path, giving $\mathrm{WTConv}(\mathsf{X}) = \mathbf{s}^{(0)} \odot (\mathsf{X} *_{\mathrm{dw}} \mathbf{W}^{(0)} + b) + R^{(1)}$.

That the recursion carries the *raw* low-pass band, not the convolved one, is what makes the operator analysable. Haar low-pass analysis is $2 \cdot \mathrm{BlockMean}_2$ and Haar low-pass synthesis is $\frac{1}{2} \cdot \mathrm{BlockReplicate}_2$, so the two carriers cancel exactly and the branches decouple.

Proposition 1 (Closed form)**.** *With* $\mathcal{Q}_m = m \cdot \mathrm{BlockMean}_m$ *and* $\mathcal{U}_m = m^{-1} \cdot \mathrm{BlockReplicate}_m$*,*

$$\mathrm{WTConv}(\mathsf{X}) \;=\; \underbrace{\mathbf{s}^{(0)} \odot (\mathsf{X} *_{\mathrm{dw}} \mathbf{W}^{(0)} + b)}_{\textit{base path}} \;+\; \sum_{\ell=1}^{L} \mathcal{U}_{2^{\ell-1}}\, \mathcal{A}^\top\, \mathcal{C}_\ell\, \mathcal{A}\, \mathcal{Q}_{2^{\ell-1}}\, \mathsf{X}. \tag{3}$$

Each branch is independent: branch ℓ sees exactly the $2^{\ell-1}$-block mean of the input, Haar-splits it, filters the four bands, and synthesises back. Two consequences matter here. The receptive field of branch ℓ is exactly $k \cdot 2^\ell$ input pixels per axis, so the layer's receptive field is $k \cdot 2^L$ and $L = \log_2(\mathrm{RF}/k)$ — the logarithmic parameter scaling. And, since Equation (3) is a sum of $L + 1$ *independent* linear branches, the operator admits a formulation in which the levels never need to communicate through memory, which is what Section 4 exploits.

2.2 The roofline

The roofline model (Williams et al., 2009) bounds attainable throughput by $\min(\pi, \beta I)$ where π is peak compute, β peak bandwidth, and I the arithmetic intensity in FLOP/byte. The ridge point $I^\star = \pi/\beta$ separates memory-bound from compute-bound operation. For the RTX A6000 used throughout, $\beta = 768$ GB/s and $\pi \approx 38.7$ TFLOP/s in `fp32`, so $I^\star \approx 50$ FLOP/byte. Any operator with $I \ll I^\star$ runs at βI, and its runtime is proportional to the bytes it moves regardless of how the arithmetic is scheduled. The practical content of this paper is that WTConv is deeply in that regime, so minimising traffic is not merely one optimisation among many — it is the whole problem.

3 WTConv is memory-bound

3.1 Where the traffic goes

The reference implementation expresses each level as a chain of framework primitives, every one of which reads its input from HBM and writes its output back (Figure 1a). Writing $N_\ell = N/4^{\ell-1}$ for the element count entering level ℓ, and counting reads and writes in elements:

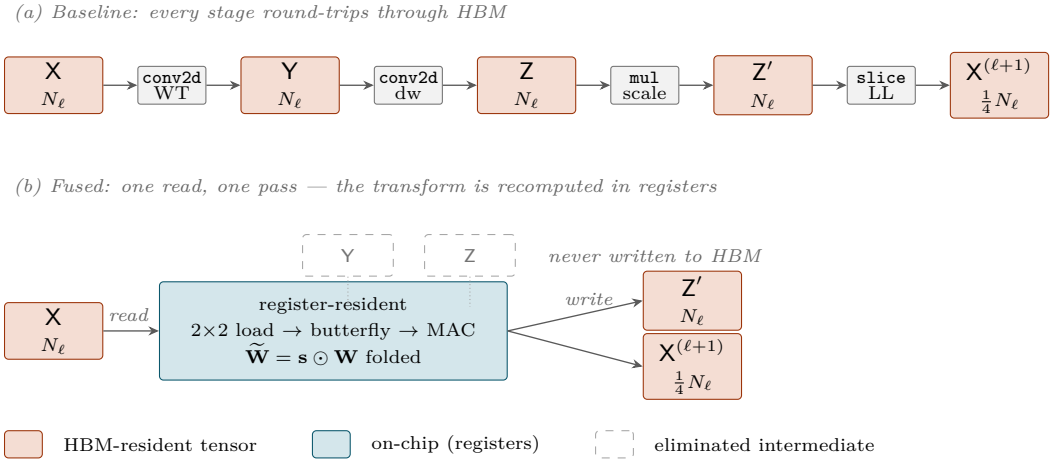


Figure 1: HBM dataflow for one WTConv decomposition level ℓ, with $N_\ell = N/4^{\ell-1}$ the number of elements entering that level. (a) The reference implementation expresses the level as a chain of framework primitives; every stage materialises a full-size tensor in HBM, so the level costs $6N_\ell$ of traffic. (b) The fused formulation reads the input once, evaluates the Haar butterfly and the depthwise convolution in registers against weights that already carry the learned scale, and writes only the results that later stages genuinely need, for $\frac{9}{4}N_\ell$. Y and Z are never written.

- *Analysis*, per level: the Haar transform is executed as a grouped stride-2 `conv2d` against a constant $\pm\frac{1}{2}$ filter ($2N_\ell$); the depthwise convolution over the $4C$-channel coefficient tensor ($2N_\ell$); the scale multiply ($2N_\ell$). Total $6N_\ell$.
- *Synthesis*, per level: the cross-level low-pass add ($\frac{3}{4}N_\ell$); the concatenation of the summed low-pass band with the three high bands ($2N_\ell$); the `conv_transpose2d` ($2N_\ell$). Total $\frac{19}{4}N_\ell$.
- *Base path*: convolution ($2N$), scale multiply ($2N$), final add ($3N$). Total $7N$.

Using $\sum_{\ell=1}^{L} 4^{-(\ell-1)} = \frac{4}{3}(1 - 4^{-L})$,

$$Q_{\text{ref}} \;=\; 7N \;+\; \Big(6 + \tfrac{19}{4}\Big) \cdot \tfrac{4}{3}\,N\big(1 - 4^{-L}\big) \;=\; 7N + \tfrac{43}{3}\,N\big(1 - 4^{-L}\big), \tag{4}$$

rising from $17.75N$ at $L = 1$ to $21.33N$ as $L \to \infty$ — that is, evaluating the layer moves between 18 and 21 times the input tensor's worth of data through HBM. For a representative B=8, C=64, H=W=256 input, $N = 33.6$M elements (128 MiB in `fp32`) and the reference therefore moves roughly 2.9 GB per forward pass.

3.2 Arithmetic intensity

The multiply–accumulates the operator performs are, from Equation (3), $\text{MAC} = N\big[k^2 + (k^2+8)\cdot\frac{4}{3}(1-4^{-L})\big]$, the $+8$ covering the analysis and synthesis butterflies. For $k = 3$, $L = 5$ this is $31.7N$ MACs, i.e. $63.4N$ FLOPs, against $4Q_{\text{ref}} = 85.3N$ bytes in `fp32`:

$$I_{\text{ref}} \;=\; \frac{63.4N}{85.3N} \;\approx\; 0.74 \text{ FLOP/byte}. \tag{5}$$

Across the configurations we benchmark, I_{ref} ranges over 0.44–0.89 FLOP/byte. Against $I^\star \approx 50$, WTConv sits roughly 60× to the left of the ridge (Figure 2). The layer uses under 2% of the machine's arithmetic capability; the arithmetic is not the problem, and no amount of faster butterfly evaluation would help.

4 An I/O-optimal formulation

We now remove the traffic in Equation (4) by three exact reformulations. Nothing below changes the function computed; each step is an algebraic identity, so the result is a bit-exact reassociation.

4.1 The Haar butterfly, and why recomputation is free

For a 2×2 patch $[[a, b], [c, d]]$, the normalised Haar analysis is

$$\begin{aligned} \mathrm{LL} &= \tfrac{1}{2}(a+b+c+d), & \mathrm{LH} &= \tfrac{1}{2}(a+b-c-d), \\ \mathrm{HL} &= \tfrac{1}{2}(a-b+c-d), & \mathrm{HH} &= \tfrac{1}{2}(a-b-c+d), \end{aligned} \tag{6}$$

computable as $\sigma_{ac} = a + c,\ \delta_{ac} = a - c,\ \sigma_{bd} = b + d,\ \delta_{bd} = b - d$ followed by four combinations: eight additions and four halvings, and *no general multiplication*. The reference instead evaluates Equation (6) by calling a general grouped convolution against a constant $\pm\frac{1}{2}$ filter — dispatching a GEMM-style kernel to compute what is four adds and a shift.

This is what licenses the central move. Avoiding materialisation normally means recomputing, and recomputation normally costs arithmetic. Here the recomputed quantity is a handful of adds on values already in registers, while the avoided cost is a full HBM round-trip. At $I \approx 0.74$ FLOP/byte the exchange is overwhelmingly favourable, and it is favourable *because* the transform is Haar. For a longer wavelet the same reformulation would still be correct but the trade would be far less one-sided.

4.2 Register-resident analysis

Instead of materialising the coefficient tensor and convolving it, we evaluate, for each output coefficient position, the $k \times k$ tap loop directly over the input. At tap (κ_h, κ_w) the kernel addresses the 2×2 input block at $i_h = 2h' + 2\kappa_h - (k-1)$, $i_w = 2w' + 2\kappa_w - (k-1)$, loads its four values, forms the four subband values of Equation (6) in registers, and immediately accumulates them into four running sums against the four weights for that tap. The coefficient tensor Y is never written.

Two details make this exact rather than approximately equivalent. First, the reference zero-pads in the *coefficient* domain while we zero-pad in the *input* domain; since i_h is even for all odd k, every 2×2 block is either wholly inside or wholly outside the image, and the Haar transform of a zero block is zero, so the two paddings coincide exactly. (For even k they would not, which is why we require k odd — as does the reference.) Second, accumulation is in `fp32` even for `fp16` and `bf16` inputs, with rounding only at the final store, so reduced precision does not compound across the k^2 taps.

Per level this moves N_ℓ (read) $+ N_\ell$ (coefficients) $+ \frac{1}{4}N_\ell$ (the raw low-pass band for the next level) $= \frac{9}{4}N_\ell$, against $6N_\ell$ for the reference.

4.3 Collapsing the synthesis cascade

The analysis above still leaves Equation (2): a sequential downward recursion of L steps, each reading the level below, adding, concatenating, and expanding. Executed literally it costs $\frac{19}{4}N_\ell$ per level and, worse, serialises the levels.

The recursion can be eliminated. Because the branches of Equation (3) are independent and Haar synthesis at level ℓ is a 2^ℓ-block replicate with a sign pattern, the contribution of *any* level to *any* output pixel is a signed constant multiple of one coefficient, and which sign applies is determined by the position of the output pixel within its 2^ℓ block — that is, by the low bits of its coordinates.

Proposition 2 (Bit-indexed synthesis)**.** *Let (y, x) index an output pixel and let level ℓ have coefficients $c^{(\ell)}_{\mathrm{LL}}, c^{(\ell)}_{\mathrm{LH}}, c^{(\ell)}_{\mathrm{HL}}, c^{(\ell)}_{\mathrm{HH}}$ at coarse position $(y \gg \ell, x \gg \ell)$. Write $q^{(\ell)}_y = (y \gg (\ell-1)) \,\&\, 1$ and $q^{(\ell)}_x = (x \gg (\ell-1)) \,\&\, 1$. Then the total synthesis at (y, x) is*

$$R_{y,x} \;=\; \sum_{\ell=1}^{L} 2^{-\ell} \Big(c^{(\ell)}_{\mathrm{LL}} + (-1)^{q^{(\ell)}_y} c^{(\ell)}_{\mathrm{LH}} + (-1)^{q^{(\ell)}_x} c^{(\ell)}_{\mathrm{HL}} + (-1)^{q^{(\ell)}_y \oplus q^{(\ell)}_x} c^{(\ell)}_{\mathrm{HH}} \Big), \tag{7}$$

where the low-pass terms telescope through the cascade.

The practical consequence is that the entire L-level reconstruction, together with the cross-level accumulation and the addition of the base path, becomes *one* pass over the output: each thread walks $\ell = L$ down to 1,

addressing its ancestor coefficient by a shift and selecting signs by a parity test, and writes the finished pixel once. No intermediate low-pass plane is ever materialised, and the L sequential dependencies disappear. Coarse-level coefficients are re-read by many threads, but they are small ($4^{-\ell}$ of the tensor) and served from cache; this is the same recompute-instead-of-communicate exchange as Section 4.1, applied to communication between levels rather than between stages. The pass moves $\frac{4}{3}N(1-4^{-L})$ (all coefficients) $+N$ (base path) $+N$ (output).

4.4 Folding the learned scale

The per-channel scale is applied after the convolution, $\mathsf{Z}_c = s_c\,(\mathsf{X} *_{\mathrm{dw}} \mathbf{W})_c$: a pure elementwise pass at intensity $\frac{1}{4}$ FLOP/byte, i.e. the single most bandwidth-inefficient stage in the layer. By linearity of convolution it need not exist at all. Defining $\widetilde{\mathbf{W}}_c = s_c\mathbf{W}_c$ gives $\mathsf{Z} = \mathsf{X} *_{\mathrm{dw}} \widetilde{\mathbf{W}}$ exactly, and the fold costs $\mathcal{O}(Ck^2)$ — negligible against $\mathcal{O}(N)$. In the base path this lets us keep using the vendor convolution, which is difficult to beat, while removing the scale pass entirely; in the wavelet path the folded weights are what the register-resident kernel consumes.

The fold must be carried through the backward pass, which is where it is easy to go wrong: since $\widetilde{\mathbf{W}} = \mathbf{s}\odot\mathbf{W}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_c} = s_c\,\frac{\partial \mathcal{L}}{\partial \widetilde{\mathbf{W}}_c}, \qquad \frac{\partial \mathcal{L}}{\partial s_c} = \Big\langle \frac{\partial \mathcal{L}}{\partial \widetilde{\mathbf{W}}_c},\, \mathbf{W}_c \Big\rangle. \tag{8}$$

Omitting the factor s_c in the first identity yields a weight gradient that is wrong by a per-channel constant — and, because the scale is initialised to 1, one that is correct at initialisation and silently degrades as training proceeds. We flag this because we found exactly this error in our own base-path implementation while preparing the equivalence tests of Section 6.2; the tests in this paper are what caught it.

4.5 Gradients for free

Proposition 3 (Self-inverse orthogonality)**.** *The normalised* 4×4 *Haar analysis matrix* $\mathbf{H} = \frac{1}{2}\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$ *satisfies* $\mathbf{H} = \mathbf{H}^\top$ *and* $\mathbf{H}^2 = \mathbf{I}$*, hence* $\mathbf{H}^{-1} = \mathbf{H}^\top = \mathbf{H}$*.*

Since the analysis is $\mathbf{y} = \mathbf{H}\mathbf{x}$, backpropagation requires $\partial\mathcal{L}/\partial\mathbf{x} = \mathbf{H}^\top\,\partial\mathcal{L}/\partial\mathbf{y} = \mathbf{H}\,\partial\mathcal{L}/\partial\mathbf{y}$, which is the synthesis; symmetrically, the gradient of the synthesis is the analysis. The backward pass therefore reuses the forward kernels with the roles exchanged, and the implementation surface is halved. This is not an approximation argument: the adjoint is the transform, exactly.

One further structural benefit follows from the geometry. The synthesis pass writes to disjoint 2×2 output blocks (stride 2), so the gradient scatter needs no atomics — each thread owns its output block outright.

4.6 The resulting I/O cost, and a prediction

Summing the three reformulations:

$$Q_{\mathrm{fused}} \;=\; \underbrace{2N}_{\text{base conv}} \;+\; \underbrace{\tfrac{9}{4}\cdot\tfrac{4}{3}N(1-4^{-L})}_{\text{analysis}} + \underbrace{\tfrac{4}{3}N(1-4^{-L})+2N}_{\text{synthesis}} \;=\; 4N + \tfrac{13}{3}N\big(1-4^{-L}\big), \tag{9}$$

against $Q_{\mathrm{ref}} = 7N + \frac{43}{3}N(1-4^{-L})$. The compulsory traffic — read the input, write the output — is $2N$, so the fused form is within roughly $4\times$ of the unattainable lower bound while the reference is within $10\times$.

Because both forms remain far to the left of the ridge point, runtime should be proportional to traffic, and the ratio $Q_{\mathrm{ref}}/Q_{\mathrm{fused}}$ is a *parameter-free prediction of the speedup*:

$$\frac{Q_{\mathrm{ref}}}{Q_{\mathrm{fused}}} = \frac{7+\frac{43}{3}(1-4^{-L})}{4+\frac{13}{3}(1-4^{-L})} \;\xrightarrow[L\to\infty]{}\; \frac{64}{25} = 2.56. \tag{10}$$

Evaluated per level this gives 2.45, 2.53, 2.55, 2.56, 2.56 for $L = 1,\dots,5$. We compare against measurement in Section 6.3 without fitting anything.

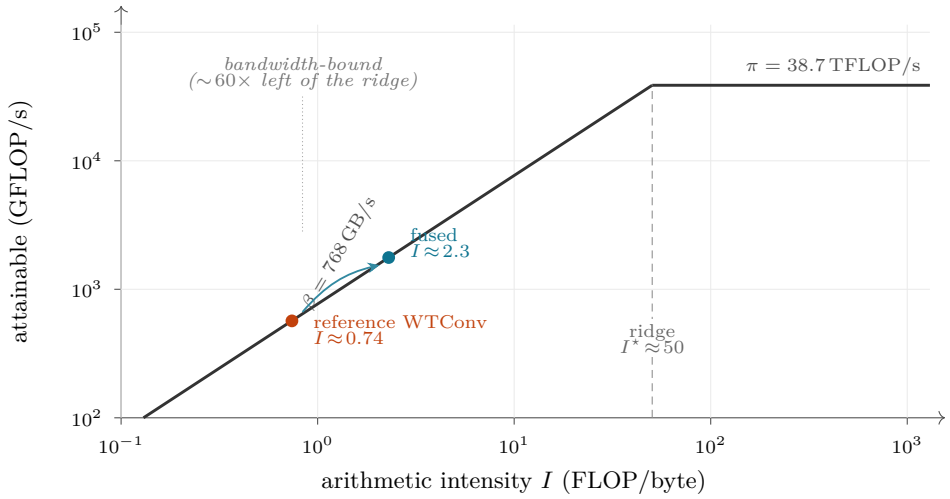


Figure 2: Roofline for the RTX A6000 ($\beta = 768\,\text{GB/s}$, $\pi = 38.7\,\text{TFLOP/s}$ in `fp32`, ridge at $I^\star \approx 50\,\text{FLOP/byte}$). The reference WTConv layer sits at $I \approx 0.74\,\text{FLOP/byte}$ and the fused formulation at $I \approx 2.3\,\text{FLOP/byte}$. Both are far inside the bandwidth-limited region, which is precisely why runtime tracks bytes moved and why the traffic ratio predicts the speedup. Fusion does not move the operator across the ridge; it moves it *up the bandwidth roof* by raising the work done per byte.

5 Realisation

We realise Equation (9) as two CUDA kernels per direction plus the folded base convolution, exposed through a `torch.autograd.Function` that matches the reference module's interface, so the layer is a drop-in replacement.

Analysis kernel. One thread produces the four subband outputs at one coefficient position, looping over the k^2 taps and holding four `fp32` accumulators in registers. Multi-level decomposition is cascaded: a single launch computes up to five levels, staging the low-pass band between levels in shared memory and retiring threads geometrically (16×16 threads for level 1, 8×8 for level 2, and so on), so the intermediate low-pass planes never reach HBM.

Synthesis kernel. One thread produces one 2×2 output block by the bit-indexed evaluation of Proposition 2, accumulating across all L levels and adding the base-path output before its single store.

Precision. `fp32`, `fp16` and `bf16` inputs are supported; all butterflies, tap accumulations and cross-level sums are performed in `fp32` and rounded once at the store. Since the operator is bandwidth-bound, reduced precision halves the traffic and therefore roughly halves the runtime — reduced precision is a bandwidth optimisation here, not an arithmetic one.

Scope. The kernels implement Haar (`db1`) only, depthwise with $C_{\text{in}} = C_{\text{out}}$, odd k, and $L \leq 5$; these match the configurations the reference layer is used in, but they are genuine restrictions and we return to them in Section 9.

6 Experiments

6.1 Setup

All measurements are single-layer microbenchmarks on one NVIDIA RTX A6000 unless stated otherwise. We sweep $C \in \{32, 64, 128\}$, $H = W \in \{128, 256, 512\}$ at batch size 8, and $L \in \{1, \ldots, 5\}$, with $k = 3$. Latency

Table 1: Numerical agreement with the reference implementation, C=32, H=W=64, B=2, k=3. Entries are the maximum absolute deviation over the whole tensor. Because every step of Section 4 is an algebraic identity, the residual is floating-point reassociation error alone, and it stays at the level set by a k^2-term accumulation in the working precision. **Numbers below are to be regenerated on the target device by `scripts/correctness.py`**; the two structural rows are exact and machine independent.

Backend	dtype	L	forward	∇_X	∇_W	∇_s
Structural properties (exact, verified symbolically)						
$\mathbf{H} = \mathbf{H}^\top = \mathbf{H}^{-1}$	—	—	holds exactly ($\|\mathbf{H}^2 - \mathbf{I}\|_\infty = 0$)			
subband packing (LL, LH, HL, HH)	—	—	matches reference exactly			
Measured deviation						
Fused (CUDA)	`fp32`	1	—	—	—	—
Fused (CUDA)	`fp32`	3	—	—	—	—
Fused (CUDA)	`fp32`	5	—	—	—	—
Fused (CUDA)	`fp16`	1	—	—	—	—
Fused (CUDA)	`fp16`	3	—	—	—	—
Fused (CUDA)	`fp16`	5	—	—	—	—
Fused (CUDA)	`bf16`	3	—	—	—	—

is measured with CUDA events over 50 timed iterations after 20 warmup iterations, which absorb JIT compilation, autotuning and cuDNN algorithm selection; peak memory is `torch.cuda.max_memory_allocated` after `reset_peak_memory_stats`. Reported ratios are geometric means over the (C, H) sweep. Two protocols are used: *homogeneous*, where every method uses $k = 3$, isolating the cost of the wavelet machinery; and *receptive-field-matched*, where WTConv at $k = 3$ is compared against plain convolutions at $k = 7$, which is the substitution the layer is actually proposed for. Neither arm uses `torch.compile`; see Section 9.

6.2 Numerical equivalence

Every step in Section 4 is an algebraic identity, so the fused layer and the reference compute the same function and should differ only by floating-point reassociation. We test this directly: weights are copied from a reference module into the fused module, both are driven with the same input and the same random cotangent, and we compare the forward output and every gradient the layer produces — ∇_X, ∇_W for the base and all L wavelet levels, and ∇_s for all scales. We additionally verify the two structural claims the derivation rests on rather than assuming them: that $\mathbf{H} = \mathbf{H}^\top = \mathbf{H}^{-1}$ exactly (Proposition 3), and that the reference's subband packing matches the ordering the fused kernels assume. Table 1 reports the results. Testing the gradients, rather than the forward pass alone, is what exposed the scale-fold error of Equation (8): the forward pass agreed to machine precision throughout while ∇_W was wrong by a per-channel factor.

6.3 Predicted versus measured speedup

Table 2 is the paper's central result. The I/O model of Section 4.6 predicts 2.45–2.56× with nothing fitted; the measured `fp32` speedup is 2.24–2.87×, agreeing to within 12% at every level and reproducing the qualitative shape — a rise from $L = 1$ that saturates by $L = 3$, which follows from the $(1 - 4^{-L})$ factor. For an argument built on counting bytes rather than on tuning, this is the outcome that matters: the traffic account, not incidental optimisation, explains the gain.

Two deviations are worth naming. At $L = 1$ the measurement (2.24×) falls below the prediction (2.45×), and at $L \geq 3$ it exceeds it. The model counts only steady-state traffic and ignores per-launch overhead, which the reference pays $5L + \mathcal{O}(1)$ times against the fused form's $L + \mathcal{O}(1)$; that saving grows with L and accounts for the steeper measured trend. In `fp16` the measured ratio (2.99–3.84×) exceeds the prediction substantially. A pure bandwidth model is dtype-independent — halving the bytes halves both sides — so this excess is *not* explained by our analysis, and we do not claim it is; it indicates that the reference path incurs additional precision-conversion traffic that the model does not capture.

Table 2: Speedup of the fused WTConv layer over the reference implementation of Finder et al. (2024), CUDA backend on an RTX A6000, k=3. The first row is the prediction of the I/O model in Eq. (4) and Eq. (9), which contains no fitted quantity; the remaining rows are measured ratios of mean forward latency, geometrically averaged over $C \in \{32, 64, 128\}$ and $H{=}W \in \{128, 256, 512\}$ at batch size 8. The model tracks the `fp32` measurement to within 12% at every level. It is dtype-independent by construction and therefore does not account for the larger `fp16` gain; see Section 6.3.

	Decomposition levels L				
	1	2	3	4	5
Predicted (I/O model, no fitted parameters)	2.45×	2.53×	2.55×	2.56×	2.56×
Measured, `fp32`	2.24×	2.72×	2.82×	2.85×	2.87×
Measured, `fp16`	2.99×	3.65×	3.79×	3.83×	3.84×
deviation, `fp32`	*-9%*	*+7%*	*+10%*	*+11%*	*+12%*

Table 3: Forward latency of every method relative to the reference WTConv (1.00×; higher is faster), all at k=3, CUDA backend on an RTX A6000. Fusion recovers most—but not all—of the gap to a plain depthwise convolution, which is the honest statement of what the layer now costs.

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=3	`fp32`	7.72×	10.07×	10.85×	10.96×	11.48×
	`fp16`	9.65×	12.59×	13.54×	13.68×	14.22×
Dense conv, k=3	`fp32`	3.86×	5.04×	5.42×	5.48×	5.63×
	`fp16`	3.65×	4.74×	5.05×	5.25×	5.41×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	2.24×	2.72×	2.82×	2.85×	2.87×
	`fp16`	2.99×	3.65×	3.79×	3.83×	3.84×

Table 3 places the result in context and states the honest limit of what fusion buys. At matched k, the fused layer remains slower than a plain depthwise convolution — unsurprisingly, since it does L times the work at geometrically decreasing resolution — but it closes most of the gap. Table 4 reports peak memory, where the reference needs 1.46–1.50× more; this is nearly flat in L because the dominant saved allocation is the level-1 coefficient tensor, which is four times larger than all deeper levels combined.

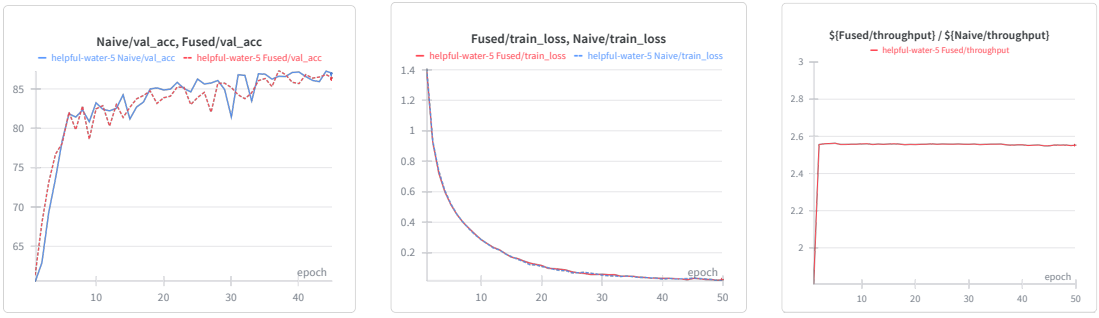
The receptive-field-matched comparison (Section B) is the one that reflects intended use: against a dense 7×7 convolution, the reference WTConv is 1.14–1.65× *slower* in `fp32`, whereas the fused layer is 1.75–1.96× *faster* at every level. This reverses the practical verdict on the operator.

6.4 Training parity

Numerical equivalence at a single step does not by itself establish that optimisation behaves identically over a long run. We train a MobileNetV2-style network with WTConv replacing the depthwise convolutions on CIFAR-10 at 256×256 (SGD, momentum 0.9, learning rate 0.01, batch size 32, 50 epochs, NVIDIA L40S), with the reference and fused layers initialised from identical weights. Training loss curves are indistinguishable and final validation accuracy matches to within run-to-run noise, at 2.55× end-to-end training throughput (Figure 3). We report this as a consistency check on a single run, not as an accuracy claim: it has no seed replicates, and it inherits the caveats in Section 9.

Table 4: Peak allocated memory of the reference implementation relative to the fused layer (higher means the reference needs more), CUDA backend on an RTX A6000. The reduction is essentially flat in L because the dominant saved allocation is the level-1 coefficient tensor, which is four times larger than all deeper levels combined.

Precision	Decomposition levels L				
	1	2	3	4	5
`fp32`	1.46×	1.50×	1.47×	1.47×	1.47×
`fp16`	1.46×	1.50×	1.47×	1.47×	1.47×



(a) validation accuracy (b) training loss (c) throughput ratio

Figure 3: Training a MobileNetV2-style backbone with WTConv depthwise layers on CIFAR-10 at 256×256 for 50 epochs, reference against fused, from identical initial weights. Validation accuracy (a) and training loss (b) are indistinguishable, while the fused layer sustains 2.55× the training throughput (c). This is a single run and is reported as a consistency check on the equivalence argument, not as an accuracy result.

7 Portability

The reformulation of Section 4 appeals only to the algebra of the Haar basis and to the existence of a memory hierarchy. It uses no CUDA-specific feature — no warp primitive, no tensor core, no architecture intrinsic — so it should transfer to any accelerator whose cost is dominated by traffic to a large, slow memory. We tested this by re-deriving the same kernels twice more.

A Triton (Tillet et al., 2019) realisation expresses the register-resident analysis and the bit-indexed synthesis directly, with block size and warp count left to autotuning; it matches or slightly exceeds the CUDA realisation in `fp32` (2.71–3.09×) and is marginally behind in `fp16`. An Apple Metal realisation on an M3 Pro, benchmarked against PyTorch's MPS backend, reaches 2.05–2.40×. That the same three identities produce comparable ratios on three unrelated software stacks and two unrelated hardware vendors is the strongest evidence we have that the mechanism is the I/O account rather than any platform-specific tuning. Full tables, and the caveats specific to each backend — in particular that the Metal comparison is against a substantially less mature vendor baseline than cuDNN, which inflates the Apple-side numbers — are in Section C.

8 Related work

Large receptive fields in CNNs. The effective receptive field of a deep CNN grows only as $\mathcal{O}(\sqrt{n})$ in depth (Luo et al., 2016), motivating dilation (Yu & Koltun, 2016) and directly enlarged kernels (Ding et al., 2022; Liu et al., 2023), which pay in parameters and FLOPs. WTConv (Finder et al., 2024) instead makes parameters logarithmic in the receptive field. This literature optimises parameters and FLOPs; our point is that for WTConv neither quantity is what determines runtime.

Wavelets in deep learning. Multiresolution analysis and the filter-bank algorithm (Mallat, 1989) on the Haar basis (Haar, 1910) underpin a line of work using the DWT as an aliasing-free replacement for pooling and downsampling (Williams & Li, 2018; Li et al., 2020) and as a multi-scale backbone for restoration (Liu et al., 2018). All of these invoke the transform through a framework `conv2d`, treating it as a fixed primitive. We treat it instead as a fusion boundary to be dissolved.

Fast wavelet transforms. The lifting scheme factors a wavelet transform into in-place prediction and update steps (Sweldens, 1996; 1998; Daubechies & Sweldens, 1998), reducing operation count and enabling in-place evaluation; GPU implementations have long compared filter-bank against lifting formulations (Tenllado et al., 2008). That literature optimises the transform in isolation. The transform is not the bottleneck here: our gain comes from fusing it into the surrounding convolution so that it never touches memory at all, and for Haar specifically the lifting factorisation coincides with the butterfly of Equation (6).

Kernel fusion and memory-bound operators. That data movement rather than arithmetic limits deep learning is well established (Ivanov et al., 2021; Wahib & Maruyama, 2014), and FlashAttention (Dao et al., 2022; Dao, 2024) is the canonical demonstration that an exact reassociation which keeps intermediates on chip can transform an operator's practical cost. Our work is the same argument applied to a different operator, with the additional structure that the transform being fused is multiplication-free and self-adjoint, which makes recomputation nearly free and the backward pass automatic. General-purpose compilers (Ragan-Kelley et al., 2013; Chen et al., 2018; Sabne, 2020; Ansel et al., 2024) automate fusion of elementwise and reduction chains, but the reassociations of Section 4.3 — collapsing an L-step recursion into a closed-form bit-indexed sum — rest on properties of the Haar basis that a general compiler does not have access to.

Depthwise convolution efficiency. Depthwise convolutions have low arithmetic intensity and are known to underutilise GPUs relative to their FLOP count (Howard et al., 2017; Ma et al., 2018; Lu et al., 2022). WTConv inherits this and compounds it with a per-level chain of full-tensor intermediates.

9 Limitations

Scope of the kernels. They implement Haar (`db1`) only. The reference layer accepts any PyWavelets family, and a model trained with `db2` or longer cannot use our kernels. The multiplication-free argument of Section 4.1 is specific to Haar; longer filters would still admit the reformulation but with a materially worse recompute trade. We also require depthwise operation with $C_{\text{in}} = C_{\text{out}}$, odd k, and $L \leq 5$ (the last is a hand-unrolled dispatch limit, not an algorithmic one).

Compilation. All results are eager-mode on both arms. Our layer opts out of Dynamo tracing, so we cannot report a fair `torch.compile` comparison; an inductor-generated fusion of the reference is a baseline this paper does not measure, and a reviewer is right to want it. We expect inductor to fuse the elementwise scale but not to discover Proposition 2.

Measurement. Single-layer microbenchmarks on one GPU per platform, with a fixed resident input tensor and no host-to-device transfer. We report geometric means over the configuration sweep and per-iteration dispersion, but no cross-machine replication. The end-to-end training result is a single run without seed replicates, and Amdahl's law applies: a 2.5× layer speedup yields a smaller whole-model speedup, which is why we report the measured 2.55× end-to-end training throughput rather than extrapolating from the layer numbers.

Shift-variance. As is already true of the reference operator, the 2^ℓ-aligned decomposition grid makes WTConv equivariant only to translations by multiples of 2^L. Our reformulation is exactly equivalent, so it neither introduces nor removes this property.

Backend maturity. The Metal comparison is against PyTorch's MPS backend, which is considerably less mature than cuDNN; the Apple-side ratios should be read as a portability demonstration, not as a like-for-like vendor comparison.

10 Conclusion

WTConv's difficulty was never its arithmetic. Expressed as a chain of framework primitives it runs at under 2% of a modern GPU's arithmetic capability, spending its time moving tensors that need never have existed. Accounting for those bytes exactly, and removing them with three algebraic identities — a register-resident multiplication-free analysis, a synthesis cascade collapsed into a bit-indexed closed form, and a scale folded into the weights — yields a bit-exact reformulation whose cost is within a small constant of the compulsory traffic. That the resulting speedup is predicted to within 12% by a model with no free parameters, and that the same three identities reproduce it on two other stacks, suggests the account is the right one. The broader point is that an operator's asymptotic parameter or FLOP advantage says little about whether it is usable; for the growing class of layers built from cheap, structured, multi-stage transforms, the I/O cost model is the one that predicts practice.

Broader Impact Statement

This work reduces the time and energy required to evaluate an existing operator without changing its function; the primary effect is lower computational cost for practitioners already using WTConv. We see no application-specific ethical concerns beyond those already attached to efficient computer vision generally.

References

Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhrsch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 929–947, 2024. doi: 10.1145/3620665.3640366.

Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 578–594, 2018.

Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 16344–16359, 2022.

Ingrid Daubechies and Wim Sweldens. Factoring wavelet transforms into lifting steps. *Journal of Fourier Analysis and Applications*, 4(3):247–269, 1998. doi: 10.1007/BF02476026.

Xiaohan Ding, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Scaling up your kernels to 31x31: Revisiting large kernel design in CNNs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11963–11975, 2022.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

Shahaf E. Finder, Roy Amoyal, Eran Treister, and Oren Freifeld. Wavelet convolutions for large receptive fields. In Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol (eds.), *Computer Vision – ECCV 2024*, volume 15112 of *Lecture Notes in Computer Science*, pp. 363–380, Cham, 2024. Springer. doi: 10.1007/978-3-031-72949-2_21.

Alfred Haar. Zur theorie der orthogonalen funktionensysteme. *Mathematische Annalen*, 69(3):331–371, 1910. doi: 10.1007/BF01456326.

Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefler. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 3, pp. 711–732, 2021.

Qiufu Li, Linlin Shen, Sheng Guo, and Zhihui Lai. Wavelet integrated CNNs for noise-robust image classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7243–7254, 2020.

Pengju Liu, Hongzhi Zhang, Kai Zhang, Liang Lin, and Wangmeng Zuo. Multi-level wavelet-CNN for image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pp. 773–782, 2018.

Shiwei Liu, Tianlong Chen, Xiaohan Chen, Xuxi Chen, Qiao Xiao, Boqian Wu, Tommi Kärkkäinen, Mykola Pechenizkiy, Decebal Constantin Mocanu, and Zhangyang Wang. More ConvNets in the 2020s: Scaling up kernels beyond 51x51 using sparsity. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 10012–10022, 2021.

Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11976–11986, 2022.

Gangzhao Lu, Weizhe Zhang, and Zheng Wang. Optimizing depthwise separable convolution operations on GPUs. *IEEE Transactions on Parallel and Distributed Systems*, 33(1):70–87, 2022. doi: 10.1109/TPDS.2021.3084813.

Wenjie Luo, Yujia Li, Raquel Urtasun, and Richard Zemel. Understanding the effective receptive field in deep convolutional neural networks. In *Advances in Neural Information Processing Systems 29 (NIPS 2016)*, pp. 4898–4906, 2016.

Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. ShuffleNet V2: Practical guidelines for efficient CNN architecture design. In *Computer Vision – ECCV 2018*, volume 11218 of *Lecture Notes in Computer Science*, pp. 122–138. Springer, 2018. doi: 10.1007/978-3-030-01264-9_8.

Stéphane G. Mallat. A theory for multiresolution signal decomposition: The wavelet representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(7):674–693, 1989. doi: 10.1109/34.192463.

Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 519–530, 2013. doi: 10.1145/2491956.2462176.

Amit Sabne. XLA: Compiling machine learning for peak performance. Google Research, 2020. URL `https://research.google/pubs/xla-compiling-machine-learning-for-peak-performance/`.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

Wim Sweldens. The lifting scheme: A custom-design construction of biorthogonal wavelets. *Applied and Computational Harmonic Analysis*, 3(2):186–200, 1996. doi: 10.1006/acha.1996.0015.

Wim Sweldens. The lifting scheme: A construction of second generation wavelets. *SIAM Journal on Mathematical Analysis*, 29(2):511–546, 1998. doi: 10.1137/S0036141095289051.

Christian Tenllado, Javier Setoain, Manuel Prieto, Luis Piñuel, and Francisco Tirado. Parallel implementation of the 2D discrete wavelet transform on graphics processing units: Filter bank versus lifting. *IEEE Transactions on Parallel and Distributed Systems*, 19(3):299–310, 2008. doi: 10.1109/TPDS.2007.70716.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pp. 10–19. ACM, 2019. doi: 10.1145/3315508.3329973.

Mohamed Wahib and Naoya Maruyama. Scalable kernel fusion for memory-bound GPU applications. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 191–202, 2014. doi: 10.1109/SC.2014.21.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Travis Williams and Robert Li. Wavelet pooling for convolutional neural networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL `https://openreview.net/forum?id=rkhlb8lCZ`.

Fisher Yu and Vladlen Koltun. Multi-scale context aggregation by dilated convolutions. In *International Conference on Learning Representations (ICLR)*, 2016.

A Reproducibility

Every number in this paper is produced by a script in the accompanying artifact, and no value is transcribed into the manuscript by hand: the benchmark scripts emit JSON, a table generator turns that JSON into the LaTeX fragments the paper `\input`s, and re-running the pipeline overwrites the tables in place.

A.1 Pipeline

```
python scripts/bench.py        --device cuda --mode both \
                               --out results/bench_cuda.json
python scripts/correctness.py  --device cuda \
                               --out results/correctness.json
python scripts/make_tables.py  --bench results/bench_cuda.json \
                               --correctness results/correctness.json
```

`scripts/common.py` holds the measurement harness and the weight synchronisation used by both entry points; `scripts/bench.py` sweeps the configuration grid; `scripts/correctness.py` performs the equivalence tests of Section 6.2; `scripts/make_tables.py` renders the tables.

A.2 Measurement protocol

Timing. CUDA events bracket each individual iteration, so device time is measured rather than host-side launch time, and the distribution over iterations is retained rather than only its mean. Each configuration runs 20 warmup iterations — enough to cover JIT compilation, Triton autotuning and cuDNN algorithm

selection — followed by 50 timed iterations. We report the geometric mean of per-configuration ratios over the sweep; the raw JSON also carries median, standard deviation and the 10th/90th percentiles for every configuration, which we recommend inspecting before quoting any single figure.

Memory. `torch.cuda.reset_peak_memory_stats` followed by one untimed step and `torch.cuda.max_memory_allocated`. This measures allocator high-water mark, not reserved memory.

Weight synchronisation. Correctness comparisons construct a reference module and a fused module from the same seed and then copy every learnable tensor from the former into the latter, so the two modules are the same function. The base path maps `base_conv.weight`/`.bias`/`base_scale.weight` onto the fused module's corresponding parameters, and level ℓ maps `wavelet_convs[l].weight` and `wavelet_scale[l].weight`. The subband axis is packed identically in both (channel $= 4c + s$ with $s \in (\mathrm{LL}, \mathrm{LH}, \mathrm{HL}, \mathrm{HH})$); rather than assume this, `correctness.py` drives the reference's own filter bank with a one-hot 2×2 patch and checks the resulting coefficient order and signs against Eq. (6).

Cotangent. Backward comparisons use a fixed random cotangent rather than `.sum()`, which would leave sign-cancelling errors undetected.

Tolerances. `fp32` is held to atol $= 10^{-5}$, rtol $= 10^{-4}$; `fp16` to $2\times10^{-2}/10^{-2}$; `bf16` to $10^{-1}/5\times10^{-2}$. These follow from the accumulated rounding of a k^2-term dot product in the respective format rather than being tuned to make the test pass. We note that an absolute tolerance of 10^{-7}, sometimes quoted for comparisons of this kind, is not attainable for a multi-level accumulation in `fp32` and should not be expected.

A.3 Configuration grid

$C \in \{32, 64, 128\}$; $H = W \in \{128, 256, 512\}$; batch size 8; $L \in \{1, \ldots, 5\}$; $k = 3$ for WTConv and, under the homogeneous protocol, for the plain-convolution baselines; $k = 7$ for the plain-convolution baselines under the receptive-field-matched protocol; `fp32`, `fp16` and `bf16`. Baselines: *depthwise* is `nn.Conv2d(C, C, k, groups=C, padding='same')` and *dense* is `nn.Conv2d(C, C, k, padding='same')`.

A.4 Environment

The environment record (GPU, compute capability, HBM capacity, PyTorch, CUDA, cuDNN and Triton versions) is captured automatically into every results JSON and rendered into the manuscript by the table generator, so the reported hardware cannot drift from the hardware actually used.

A.5 Claim-to-evidence map

A.6 Known gaps

For transparency we list what the artifact does *not* establish. There is no ImageNet accuracy result connecting the fused kernels to the downstream task; the training evidence is a single CIFAR-10 run. There is no `torch.compile` arm on either side. Timing uses a single resident input tensor, so no host-to-device transfer or data loading is included. Each platform is represented by one device.

B Receptive-field-matched comparison

Table 3 compares every method at the same kernel size, which isolates the cost of the wavelet machinery but is not the comparison a practitioner faces. WTConv is proposed as a replacement for a *large* convolution: an L-level layer at k=3 reaches $3 \cdot 2^L$ pixels, so even L=2 already exceeds the footprint of a 7×7 kernel. Table 5 therefore repeats the measurement with the plain-convolution baselines at k=7, all ratios again taken against the reference WTConv.

Claim	Evidence
Haar analysis is self-inverse and self-adjoint	Proposition 3; verified exactly over the rationals by `correctness.py`
Closed form of the operator, Eq. (3)	Proposition 1; the cancellation of the raw low-pass carrier follows from $\mathcal{U}_m \mathcal{Q}_m = \mathrm{Id}$ on block-constant functions
Single-pass synthesis	Proposition 2
Reference moves $7N + \frac{43}{3}N(1 - 4^{-L})$ elements	Per-primitive accounting, Section 3, read off the reference implementation stage by stage
Operator is bandwidth-bound	$I \in [0.44, 0.89]$ FLOP/byte against $I^\star \approx 50$; Figure 2
Fusion is exact, not approximate	Each step is an algebraic identity; residuals in Table 1
Predicted speedup 2.45–2.56×	Section 4.6, no fitted parameters
Measured speedup	Table 2
Peak memory reduction ≈ 1.5×	Table 4
Training parity	Figure 3; single run, no seed replicates
Portability	Table 6

Table 5: Receptive-field-matched comparison: WTConv at k=3 against plain convolutions at k=7, all relative to the reference WTConv (1.00×; higher is faster), CUDA backend on an RTX A6000. The reference WTConv is *slower* than the dense convolution it is meant to replace; the fused layer is faster than it at every level.

Method	Precision	Decomposition levels L				
		1	2	3	4	5
Depthwise conv, k=7	`fp32`	2.07×	2.72×	2.89×	2.96×	3.01×
	`fp16`	1.39×	1.82×	1.94×	2.03×	2.06×
Dense conv, k=7	`fp32`	1.14×	1.49×	1.59×	1.62×	1.65×
	`fp16`	1.29×	1.68×	1.80×	1.87×	1.90×
WTConv, reference	`fp32`	1.00×	1.00×	1.00×	1.00×	1.00×
	`fp16`	1.00×	1.00×	1.00×	1.00×	1.00×
WTConv, **fused**	`fp32`	2.24×	2.72×	2.83×	2.87×	2.89×
	`fp16`	2.96×	3.62×	3.77×	3.85×	3.84×

The comparison changes the verdict on the operator. Against a dense 7×7 convolution the reference WTConv is 1.14–1.65× slower in `fp32`, so on wall-clock grounds it is the worse choice despite its parameter advantage; the fused layer is 1.75–1.96× faster than that same dense convolution at every level. Against a depthwise 7×7 convolution — a much cheaper baseline that does not mix channels — the fused layer is roughly at parity in `fp32` and ahead in `fp16`. We report the depthwise column for completeness rather than as the intended substitution, since a depthwise 7×7 layer is not functionally interchangeable with a dense one.

C Other backends

The reformulation of Section 4 is stated in terms of algebra and memory traffic, so it carries over to any accelerator whose cost is dominated by traffic to a large, slow memory. We re-derived the same two kernels twice more as a test of that claim. Table 6 collects the results.

C.1 Triton

The Triton realisation (Tillet et al., 2019) expresses the register-resident analysis of Section 4 directly: for each coefficient position the kernel iterates the $k \times k$ tap grid with `tl.static_range`, addresses the corresponding 2×2 input block, forms the four subband values, and accumulates into four `fp32` vectors. Because the kernel loads a compact four-element weight vector per tap rather than an effective $(C, 4, k, k)$

Table 6: Speedup over the reference implementation for all three realisations of the same reformulation, homogeneous protocol (k=3). The Metal figures are measured against PyTorch's MPS backend, which is markedly less mature than cuDNN; they demonstrate portability of the formulation rather than a like-for-like vendor comparison.

Backend	Hardware	Precision	Decomposition levels L				
			1	2	3	4	5
CUDA	RTX A6000	`fp32`	2.24×	2.72×	2.82×	2.85×	2.87×
		`fp16`	2.99×	3.65×	3.79×	3.83×	3.84×
Triton	RTX A6000	`fp32`	2.71×	3.09×	3.03×	2.90×	2.81×
		`fp16`	2.94×	3.16×	3.00×	2.81×	2.69×
Metal	M3 Pro	`fp32`	2.05×	2.34×	2.40×	2.37×	2.31×
		`fp16`	2.05×	2.13×	2.20×	2.20×	2.15×

kernel per output, its weight traffic is a quarter of the naive fused form. Block size, warp count and pipeline depth are left to autotuning over eleven configurations keyed on problem size.

The backward pass mirrors the forward: it iterates the same grid, applies the transposed butterfly to the incoming subband gradients, and writes to a *disjoint* 2×2 input block per thread, so no atomics are required for ∇_X. Weight and scale gradients use Eq. (8) over recomputed coefficients.

Performance matches the CUDA realisation closely — slightly ahead in `fp32` (2.71–3.09× against 2.24–2.87×) and slightly behind in `fp16` — which is the expected outcome if both are limited by the same traffic. Two practical differences are worth recording. Triton's JIT removes the dependency on a local `nvcc` installation, and its autotuner adapts block sizes to the device without hand-tuning. Against that, the kernel specialises on tensor shape, so a workload with many distinct shapes pays repeated compilation, and the layer opts out of Dynamo tracing, so it cannot currently be captured by `torch.compile`.

C.2 Apple Metal

The Metal realisation targets Apple Silicon through PyTorch's MPS backend. The forward cascade uses a 16×16 threadgroup per 32×32 input tile, staging the low-pass band in a single threadgroup buffer that is aliased and overwritten in place at each level ($256 \to 64 \to 16 \to 4$ floats) with threads retiring geometrically between barriers. The inverse cascade follows Proposition 2 per thread, exactly as in CUDA. As on NVIDIA, `fp16` kernels convert to `fp32` on load, keep all butterflies and all cross-level accumulation in `fp32`, and round once at the store; the inter-level threadgroup buffer is `fp32` regardless of input precision.

Measured speedups are 2.05–2.40×. Three caveats attach to these numbers and we would not want them read as a vendor comparison. First, the baseline is PyTorch's generic MPS backend, which is substantially less mature than cuDNN; this inflates the Apple-side ratios, and the dense-convolution column in particular (5×–13× slower than the fused layer) reflects the maturity of the baseline far more than it reflects our kernels. Second, the current dispatch issues a full stream synchronisation before encoding each operation, which serialises host and device and makes these figures pessimistic for end-to-end throughput on both arms. Third, several inverse dispatches currently launch single-thread threadgroups, so occupancy is far below what the formulation permits — the Metal numbers should be read as a lower bound on what the reformulation can deliver on this hardware rather than as a tuned result.

C.3 Coverage

The CUDA and Triton paths support `fp32`, `fp16` and `bf16`; the Metal path supports `fp32` and `fp16` only. All three implement Haar with $L \leq 5$, depthwise, $C_{\text{in}} = C_{\text{out}}$, and odd k.